Projet Machine Learning Retail

Analyse du churn client, segmentation et prédiction du revenu dans le e-commerce de cadeaux

Kammoun Imen

École Nationale d'Ingénieurs de Sfax | Département GI2-MA

Année universitaire : 2025-2026

Project Complete

Exploration → Préparation → Modélisation → Évaluation → Déploiement

Plan de la Présentation

01 **Introduction & Contexte**

Problématique, objectifs, méthodologie CRISP-DM

02 **Description des Données**

Dataset : 4 320 clients, 52 features, qualité

03 **Préparation des Données**

Pipeline 8 étapes, anti data leakage, feature engineering

04 **Modélisation**

KMeans+PCA · RandomForest Churn · RandomForest Régression

05 **Résultats & Analyse**

Clusters, performances, features importance

06 **Déploiement Flask**

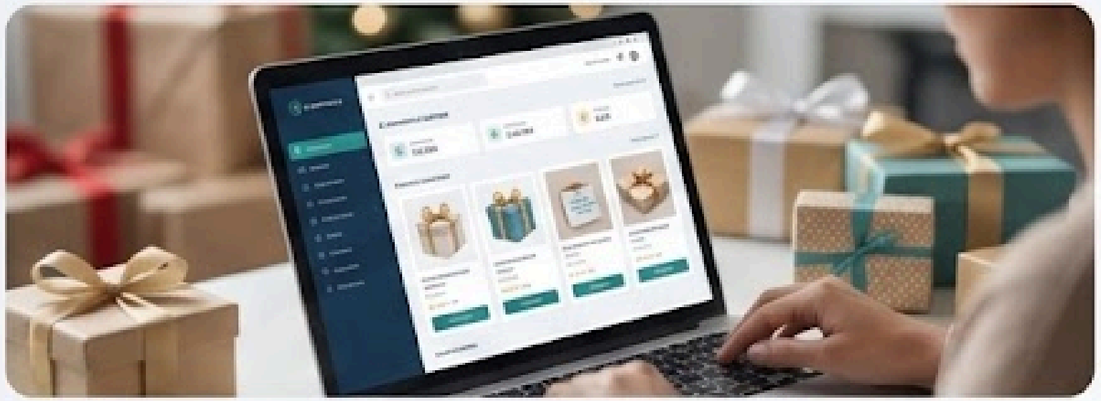
Architecture, pipeline prédiction, démonstration

07 **Conclusion**

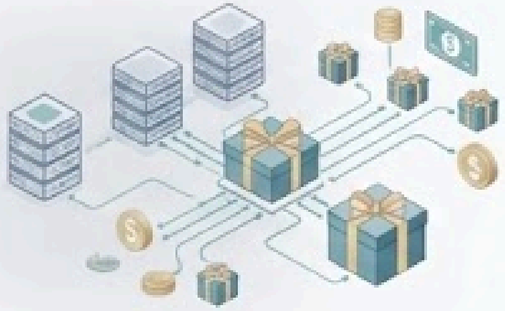
Bilan, difficultés, perspectives d'amélioration

Introduction

Secteur : e-commerce de cadeaux en ligne



- Dans le contexte du e-commerce, les entreprises génèrent **une grande quantité de données clients** issues des transactions. Ces données métier des résultats.



- Une ressource précieuse pour comprendre le **comportement des consommateurs** et **améliorer** les relations et...

  Cependant, leur exploitation reste complexe, ce qui rend nécessaires des outils pour l'information des utiles à prise de décision.

01 Introduction & Contexte

Contexte

- **Dataset réel** : e-commerce de cadeaux en ligne (UK)
- **4 320 clients** après filtrage (52 features brutes → 48)
- **Défis réels** : NaN, outliers, data leakage

Méthodologie

Approche CRISP-DM adaptée
- Split stratifié 80/20
- Anti data-leakage strict
- Optimisation Optuna

Segmentation

Identifier des groupes homogènes de clients pour personnaliser les stratégies marketing

Prédiction churn

Détecter proactivement les clients à risque de départ pour agir avant leur départ

Estimation revenu

Prédire la valeur monétaire future des clients pour optimiser les ressources commerciales

Pipeline & Méthodologie CRISP-DM

Secteur : e-commerce de cadeaux en ligne

1 Exploration

- Analyse de la qualité des données
- Étude des distributions et corrélations
- Identification des problèmes (NaN, outliers, incohérences)

2 Préparation

- Nettoyage des données
- Feature engineering
- Encodage des variables
- Split train/test (80% / 20%)

3 Modélisation

- Segmentation : KMeans + ACP
- Classification (churn) : Random Forest
- Régression (revenu) : Random Forest

4 Évaluation

- Clustering : Silhouette Score
- Classification : ROC-AUC, Accuracy
- Régression : MAE, R^2

5 Déploiement

- Développement d'une application web avec Flask
- Prédictions en temps réel :
- segment client
- risque de churn
- revenu estimé

Choix Technologiques Justifiés

Python (scikit-learn) — bibliothèque ML standard, intégration joblib pour sérialisation | **Flask** — micro-framework léger, idéal API REST ML | **Jinja2** — moteur templates intégré Flask (HTML dynamique sans JS complexe) | **Optuna** — optimisation bayésienne (> GridSearch exhaustif)

02 Description des Données

4 320
Clients (après filtrage)

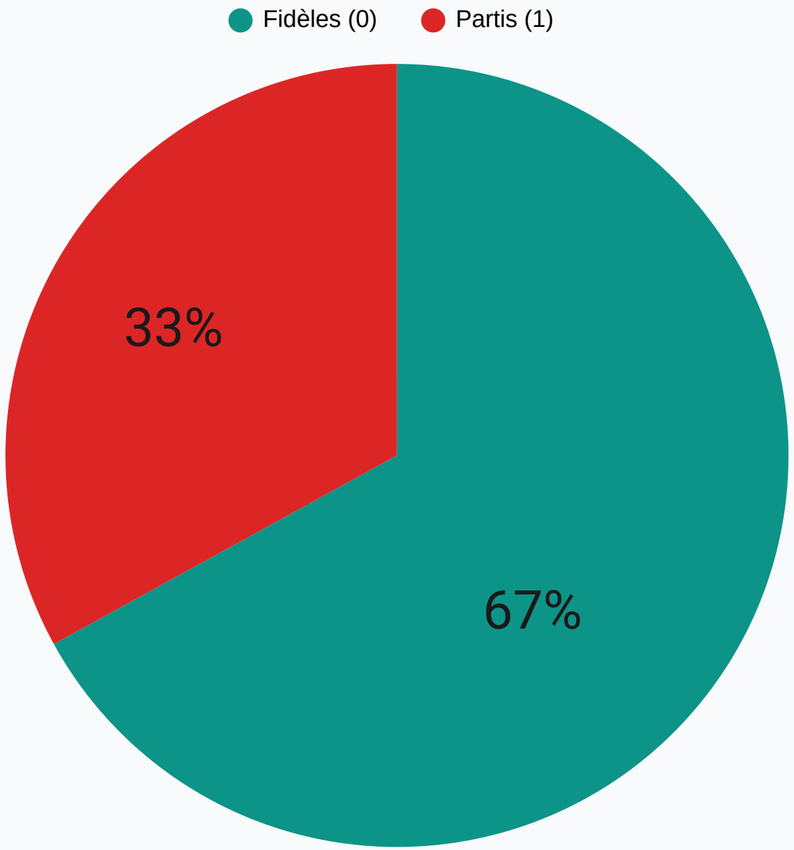
52
Features brutes

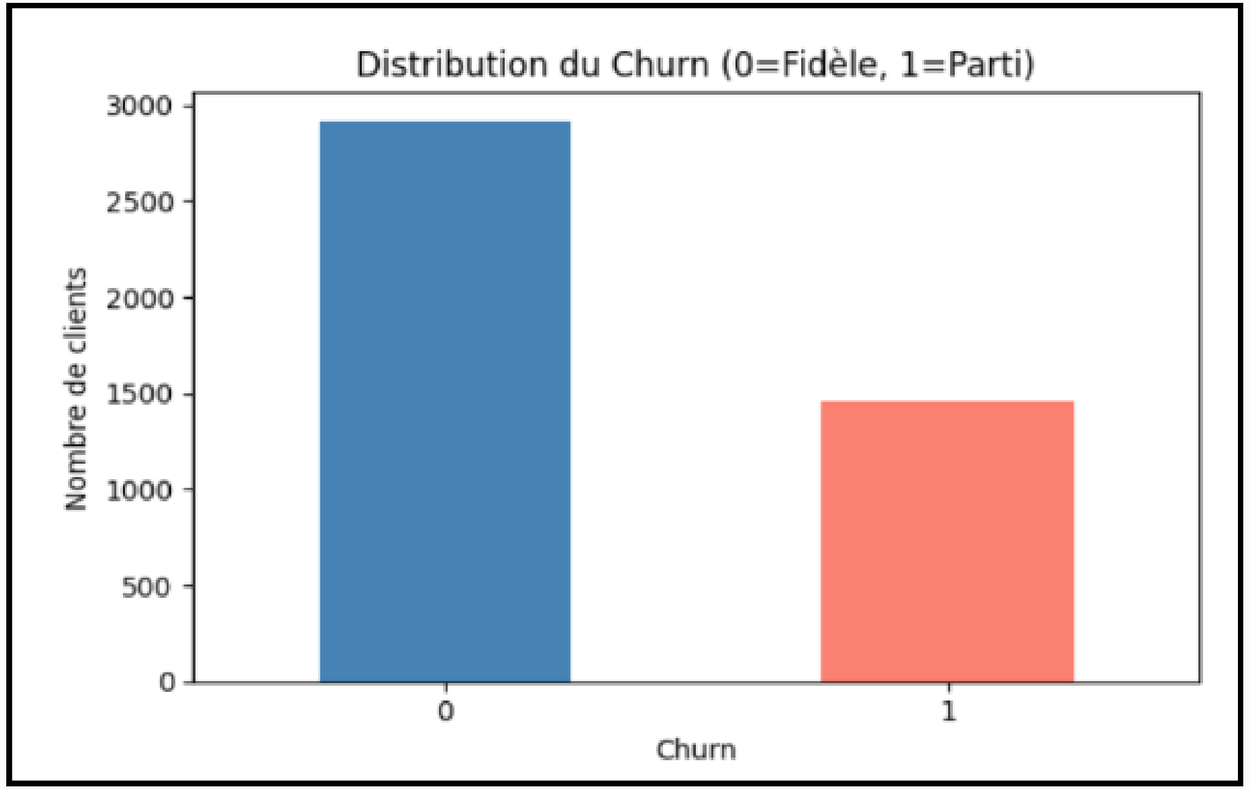
48
Features après preprocessing

33%
Taux de churn

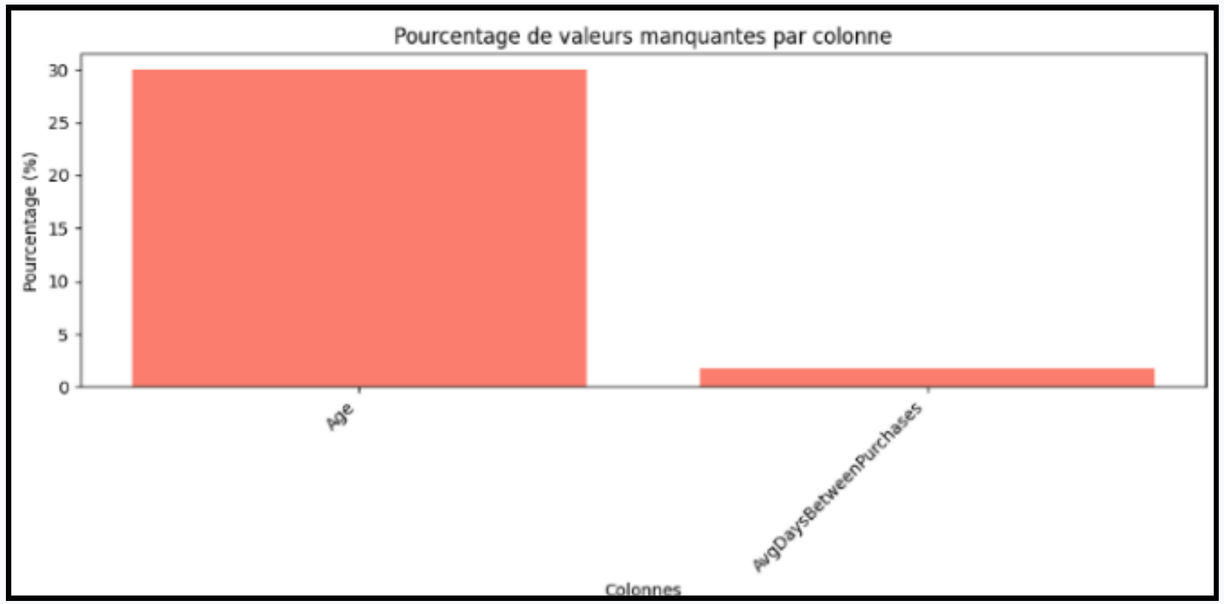
Types de features

Catégorie	Nb	Exemples
Numériques (RFM, transactions)	34	Recency, Frequency, MonetaryTotal…
Catégorielles (profils, géo, démo)	18	RFMSegment, CustomerType, Region…
Variable cible binaire	1	Churn (0 = fidèle, 1 = parti)

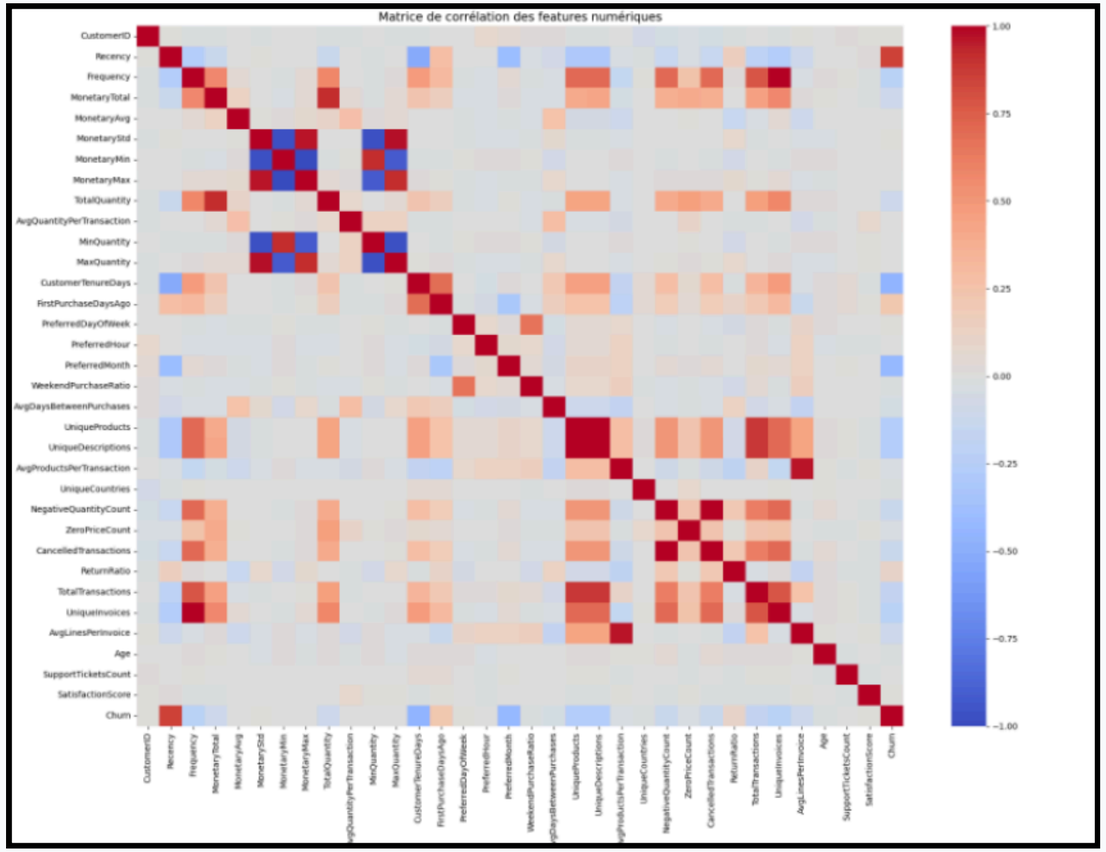




Distribution de la variable cible Churn



Visualisation des valeurs manquantes naturelles par colonne (avant correction des codes erreurs)



Matrice de corrélation des features numériques

Qualité des données — Problèmes identifiés & Traitements

Problèmes détectés lors de la phase d'exploration

Problèmes de Qualité Identifiés & Traitements

Valeurs manquantes

Age : 30% NaN → médiane train (49 ans)
AvgDaysBetween : ~2% → médiane (1.16)

Outliers déguisés

SupportTickets : -1, 999 → NaN
SatisfactionScore : -1, 99 → NaN

Formats incoh.

RegistrationDate : 3 formats mixtes
→ pd.to_datetime(format='mixed')

Data Leakage

ChurnRiskCategory calculée depuis
Churn → suppression OBLIGATOIRE

Feature inutile

NewsletterSubscribed : variance
nulle (toujours 'Yes') → supprimée

Valeurs négatives

MonetaryTotal ≤ 0 : 52 lignes
→ suppression (remboursements)

03 Préparation des Données — Pipeline 8 Étapes

1 Parsing dates & IPs

RegistrationDate → RegAncienneté
LastLoginIP → IP_privee (0/1)

2 Suppression colonnes

28 colonnes supprimées dont
ChurnRiskCategory (data leakage !)

3 Correction outliers

SupportTickets: -1, 999 → NaN
MonetaryTotal ≤ 0: 52 lignes supprimées

4 Feature Engineering

AvgBasketValue = Monetary/(Freq+1)
CancelRatio = Annulés/(Total+1)

5 Encodage catégorielles

Ordinal: PreferredTimeOfDay (0→4)
One-Hot: Gender, IP_privee
Target Enc: Country (taux churn train)

6 Split stratifié 80/20

Train: 3 456 | Test: 864
Stratifié sur Churn → équilibre classes

7 Imputation post-split

Médiane calculée sur TRAIN uniquement
Appliquée sur test → anti data leakage

8 Normalisation post-split

StandardScaler.fit_transform(X_train)
StandardScaler.transform(X_test)
24 colonnes continues (binaires exclues)

⚠ Règle Anti Data Leakage : StandardScaler fitté sur X_train UNIQUEMENT — jamais sur X_test. Target Encoding calculé sur df_train puis appliqué sur df_test. Aucun re-split dans train_model.py.

04 Modélisation 1/3 — Clustering : KMeans + PCA

Pourquoi KMeans ?

✓ **Adapté segmentation RFM**
(données métriques continues)
✓ **Centroïdes = profils clients** interprétables
✓ **Scalable** et rapide sur 3 456 observations
✓ **Combiné à PCA** pour réduire bruit et dimension

Pourquoi PCA ?

✓ **5 features → 4 composantes**
conservant 96.5% de la variance
✓ **Élimine la multicolinéarité** entre
Freq/MonetaryTotal/TotalTransactions
✓ **Visualisation 2D** des clusters possible
→ PC1 : 49% | PC2 : 23% | PC3 : 13.5% | PC4 : 11%

5 Features utilisées (RFM étendu)

Frequency · MonetaryTotal · CustomerTenureDays
AvgDaysBetweenPurchases · TotalTransactions
(Note: Recency supprimée car redondante)

Choix de K = 3 — Justification

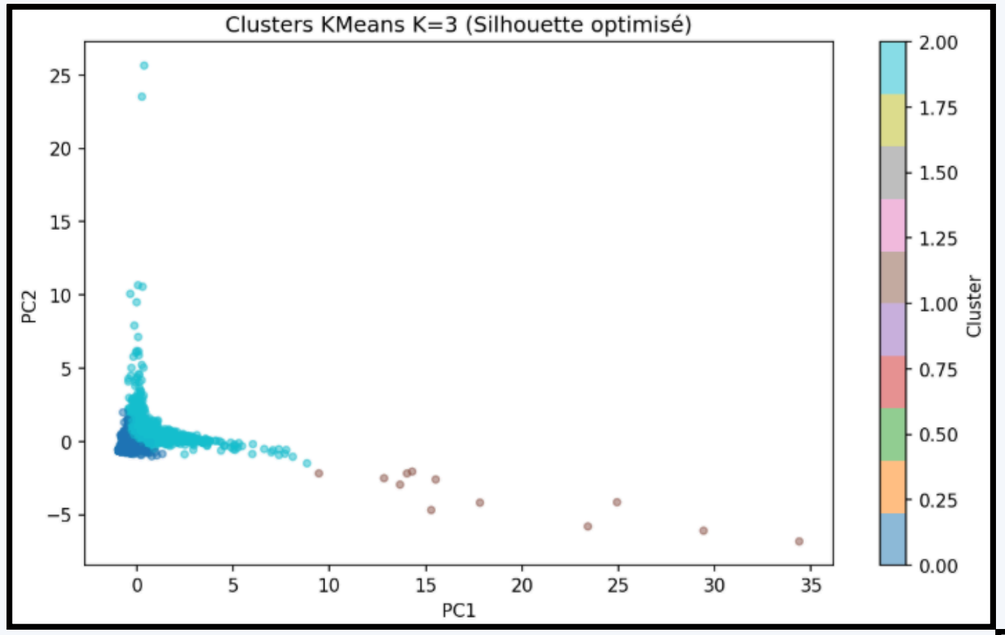
Algorithme automatique :
Test K=2 à 6 + Silhouette Score

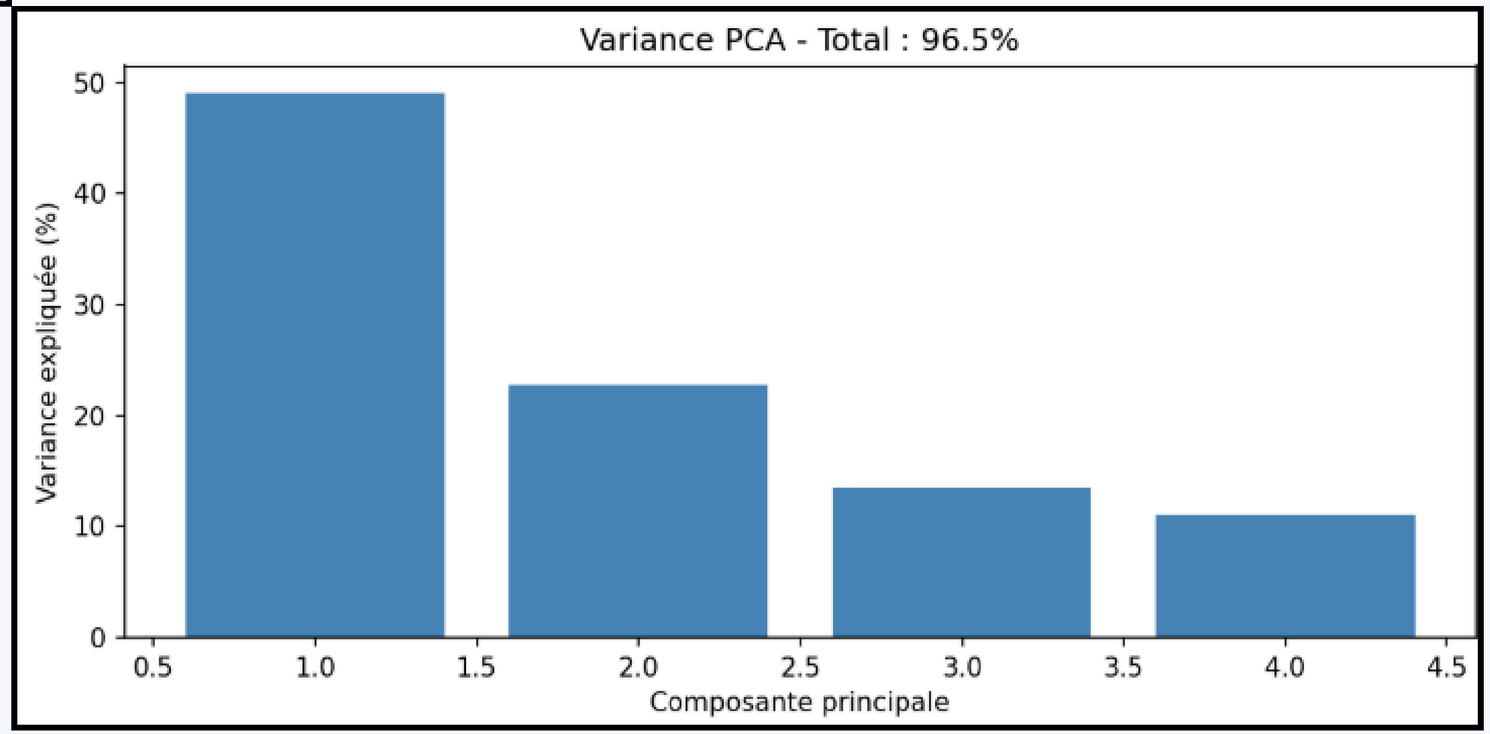
Problème détecté :
K=2 à 6 rejetés car cluster < 1%
(13 clients Ultra-VIP isolés naturellement)

Solution → Fallback K=3 :
3 segments d'interprétabilité métier maximale

Distribution finale :

Cluster 0 : 2 028 clients (58.7%)
Cluster 1 : 12 clients (0.3%) — Ultra-VIP
Cluster 2 : 1 416 clients (41.0%)


Clusters KMeans K=3 (Silhouette optimisé)
PC2
PC1
Cluster
25
20
15
10
5
0
−5
0
5
10
15
20
25
30
35
2.00
1.75
1.50
1.25
1.00
0.75
0.50
0.25
0.00


Variance PCA - Total : 96.5%
Variance expliquée (%)
Composante principale
50
40
30
20
10
0
0.5
1.0
1.5
2.0
2.5
3.0
3.5
4.0
4.5

04 Résultats du Clustering — 3 Segments Clients

Cluster	Profil	Nb Clients	Freq. moy.	Monetary moy.	Churn %	Alerte
0	Clients à Risque (Faible activité)	2 028 (58.7%)	2.5	626 £	29.4%	VIGILANCE
1	Clients Ultra-VIP (Exceptions)	12 (0.3%)	105.5	110 482 £	53.8%	CRITIQUE
2	Clients Premium (Réguliers)	1 416 (41.0%)	8.78	2 986 £	38.3%	MODÉRÉ

⚠ Cluster 1 — Paradoxe VIP

Revenu moyen 110 482 £ MAIS 53.8% de churn. La perte d'1 client = perte de centaines d'ordinaires → Programme dédié URGENT.

✓ Cluster 2 — Socle commercial

41% des clients, revenu moyen 2 986 £, ancienneté 277j. Socle stable de l'entreprise → Actions fidélisation régulières.

04 Modélisation 2/3 — Classification Churn : RandomForest + Optuna

Pourquoi Random Forest ?

✓ Ensemble d'arbres → robuste au bruit et outliers
✓ Gère données hétérogènes (num. + catég.)
✓ class_weight='balanced' pour déséquilibre 67/33
✓ Importance des features via critère Gini
✓ Peu sensible à la normalisation des features

Pourquoi Optuna vs GridSearchCV ?

GridSearch : teste TOUTES les combinaisons → lent
Optuna : optimisation bayésienne intelligente → utilise les résultats précédents pour guider la recherche
→ 30 trials, optimisation ROC-AUC en CV 3-folds

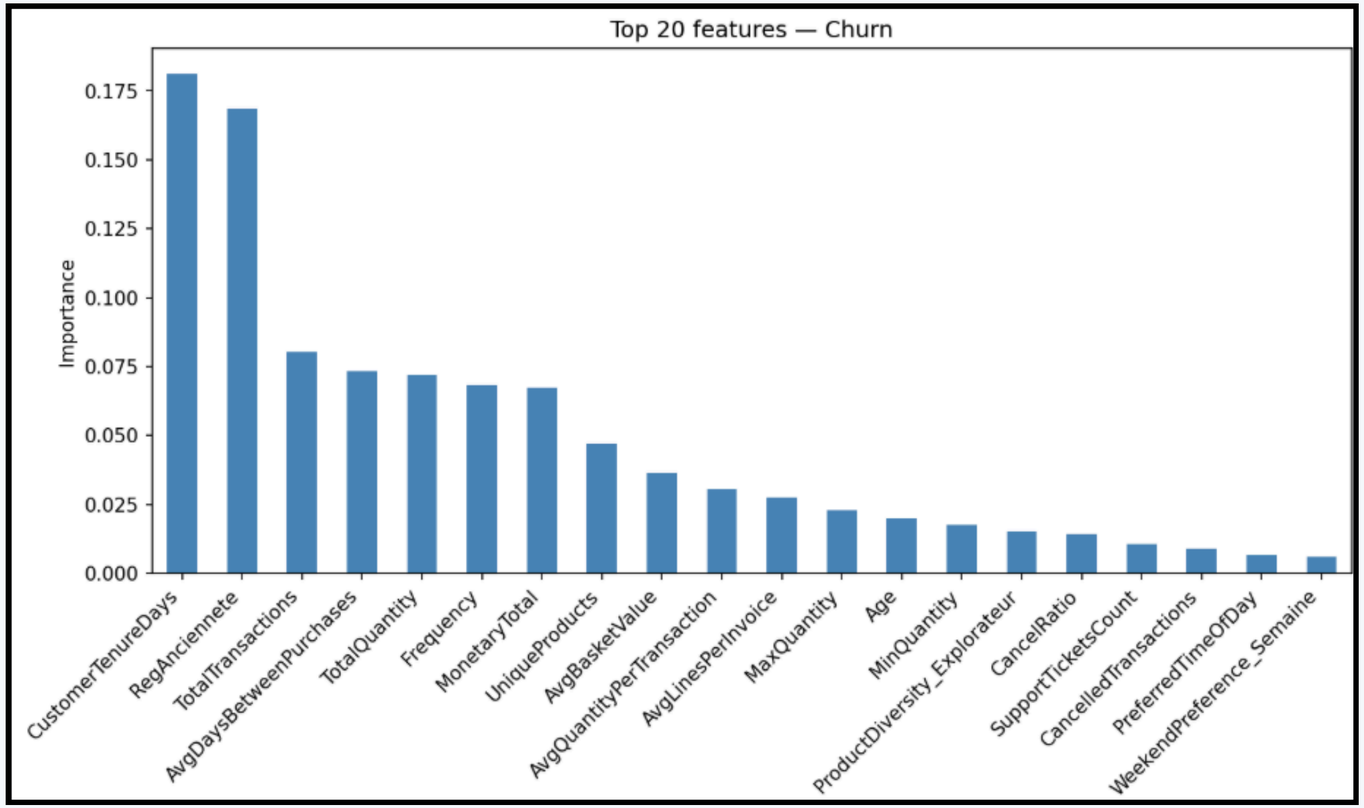
Hyperparamètres Optimaux (Optuna — 30 trials)

`n_estimators`	260	Nombre d'arbres dans la forêt
`max_depth`	12	Profondeur max (évite surapprentissage)
`min_samples_split`	8	Obs. min pour diviser un nœud
`min_samples_leaf`	6	Obs. min par feuille
`class_weight`	balanced	Compense le déséquilibre 67/33

Résultats sur jeu de test (864 observations jamais vues à l'entraînement)

0.8681	80%	88%	77%	0.71
ROC-AUC	Accuracy	Précision Fidèles	Rappel Churnés	F1 Churnés

04 Features Importantes — Prédiction du Churn



Ancienneté domine (34.9%)

CustomerTenureDays (18.1%) + RegAnciennete (16.8%) = un client ancien et inscrit depuis longtemps est MOINS susceptible de partir.

Activité transactionnelle protège

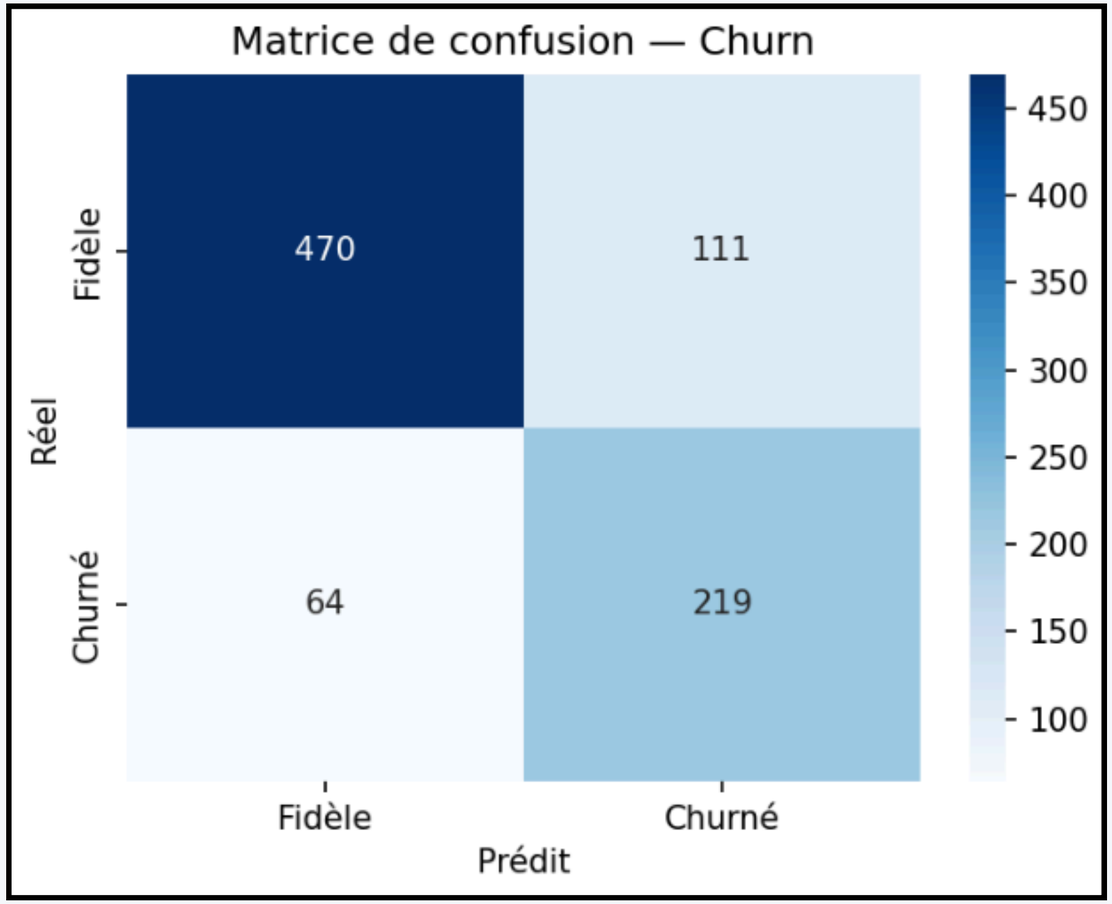
TotalTransactions (8%) + Frequency (6.8%) + TotalQuantity (7.2%) : un client qui achète régulièrement et en volume est naturellement plus fidèle.

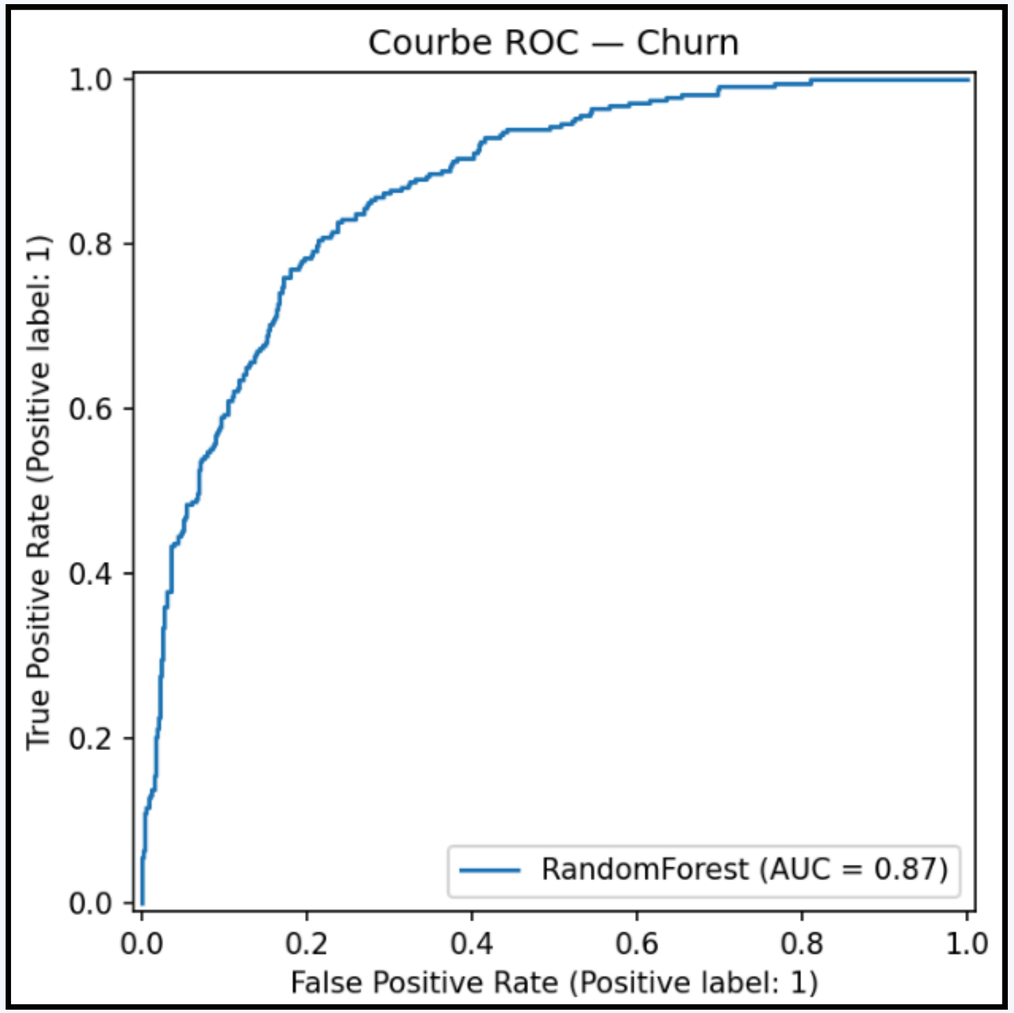
Valeur monétaire = signal secondaire

MonetaryTotal (6.7%) et AvgBasketValue (3.6%) : les clients à fort pouvoir d'achat sont légèrement plus fidèles, mais ce n'est pas le facteur dominant.

Rappel 77% sur churnés

Le modèle détecte 77% des départs réels → intervention précoce sur la majorité des cas à risque (23% de faux négatifs acceptables).


Matrice de confusion — Churn
Réel
Fidèle
Churné
470
111
64
219
Fidèle
Churné
Prédit
450
400
350
300
250
200
150
100


Courbe ROC — Churn
True Positive Rate (Positive label: 1)
False Positive Rate (Positive label: 1)
0.0
0.2
0.4
0.6
0.8
1.0
RandomForest (AUC = 0.87)

04 Modélisation 3/3 — Régression Panier Moyen : RandomForest + Optuna

Cible : AvgBasketValue = MonetaryTotal / (Frequency + 1)

Justification : MonetaryTotal brut → forte dispersion, clients VIP jusqu'à 279 489 £ → très difficile à prédire avec précision. AvgBasketValue est normalisée par la fréquence → beaucoup plus stable → MAE faible et R^2 élevé. Le +1 évite division par zéro.

Hyperparamètres (Optuna — 30 trials)

`n_estimators`	186	Nombre d'arbres
`max_depth`	14	Profondeur max
`min_samples_split`	3	Obs. min pour division
`min_samples_leaf`	2	Obs. min par feuille
`Optimisé sur`	MAE min	CV 3-folds sur X_train

Résultats sur jeu de test

MAE	15.72 £	Erreur moyenne absolue de prédiction
R^2	0.8024	80.24% de la variance du panier expliquée
Moy. prédite	226.52 £	Très proche de la moyenne réelle (225.51 £)
Écart biais	< 0.5%	Absence de biais systématique confirmée

Précision par Segment Client

Cluster 0 (Faible activité)

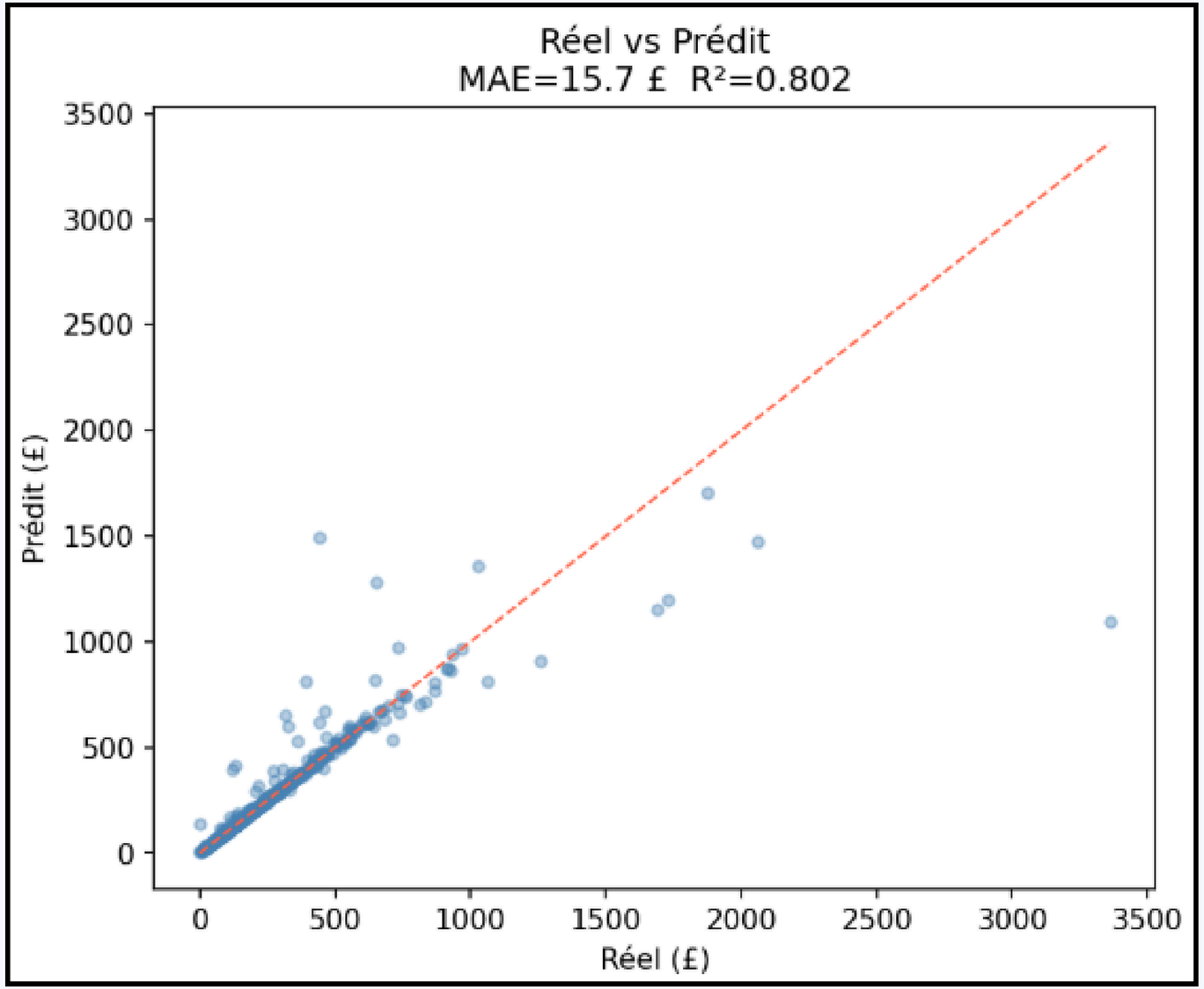
Panier: ~626 £ | 15.72 £ = ~2.5% du panier ✓

Cluster 2 (Premium)

Panier: ~2 986 £ | 15.72 £ = ~0.5% du panier ✓✓

Cluster 1 (Ultra-VIP)

Panier: ~110 482 £ | Ecarts plus importants (rareté dans l'entraînement)


Réel vs Prédit
MAE=15.7 £ R²=0.802
Prédit (£)
Réel (£)
0
500
1000
1500
2000
2500
3000
3500

04 Mesures Anti Data Leakage — Rigueur Scientifique

Définition : Le data leakage survient quand des informations du jeu de test 'contaminent' l'entraînement → performances artificiellement parfaites, inutilisables en production.

⚠ Sans mesures : Accuracy=100% grâce à ChurnRiskCategory qui contient directement la réponse ! — résultat invalide scientifiquement.

ChurnRiskCategory

⚠ Risque : Calculée à partir de Churn → donne la réponse au modèle avant même d'apprendre

→ Solution : Supprimée en 1ère étape de preprocessing.py avant tout autre traitement

StandardScaler

⚠ Risque : Si fitté sur tout le dataset → fuite des stats du test (moyenne, std) vers l'entraînement

→ Solution : fit_transform(X_train) UNIQUEMENT puis transform(X_test) — jamais l'inverse

Target Encoding Country

⚠ Risque : Si calculé avant le split → contient l'information sur y_test dans les features

→ Solution : Calculé sur df_train uniquement puis .map() appliqué sur df_test séparément

Re-split interdit

⚠ Risque : Si re-split dans train_model.py → données test pourraient apparaître dans le train

→ Solution : train.csv et test.csv chargés séparément depuis fichiers, aucun re-split effectué

Imputation post-split

⚠ Risque : Si médiane calculée avant split → utilise info du test pour imputer le train

→ Solution : Médiane calculée sur df_train.median() puis appliquée sur df_test.fillna()

Features suspectes

⚠ Risque : TenureRatio, CancelRatio, AccountStatus corrélées directement avec Churn

→ Solution : Retirées de la liste des features pour les modèles supervisés (classification & régression)

05 Bilan des Résultats & Recommandations Métier

Segmentation
KMeans + PCA

K=3 clusters
Silhouette optimisé

96.5%	3
Variance PCA	Segments métier

Classification
RandomForest + Optuna

30 trials | ROC-AUC

0.8681	80%
ROC-AUC	Accuracy

Régression
RandomForest + Optuna

AvgBasketValue | 30 trials

15.72 £	0.8024
MAE	R^2

Recommandations Business par Segment

Cluster 1 — Ultra-VIP 12 clients | 53.8% churn | Revenu ~110 482 £	Programme premium dédié, gestionnaire de compte, contact direct dès 1er signal d'inactivité	**PRIORITÉ ABSOLUE**
Cluster 0 — Faible activité 2 028 clients | 29.4% churn | Revenu ~626 £	Email réactivation J+30 sans achat, offre personnalisée, monitoring bi-mensuel automatisé	**VOLUME CRITIQUE**
Cluster 2 — Premium 1 416 clients | 38.3% churn | Revenu ~2 986 £	Newsletter ciblée, programme ambassadeur, accès anticipé ventes privées, offres anniversaire	**CONSOLIDATION**

06 Déploiement — Interface Flask

Pourquoi Flask + Jinja2 ?

✓ Micro-framework Python léger — aucune surcharge
✓ Crée des API REST pour servir modèles scikit-learn
✓ Intégration facile joblib (.pkl) → chargement unique
✓ Jinja2 intégré → templates HTML dynamiques sans JS
✓ Alternatives : FastAPI (async), Streamlit, Django (lourd)

Architecture (app/)

`app.py`	Serveur Flask — charge modèles, reçoit POST, 3 pipelines
`templates/index.html`	Interface HTML/CSS/Jinja2 — formulaire + résultats
`models/*.pkl`	10 fichiers : kmeans, pca, churn_model, scaler, columns…
`src/cluster_labels.py`	Labels centralisés — cohérence app.py / predict.py

Pipeline de Prédiction Séquentielle

Formulaire
9 features RFM
→
Norm. + PCA
+ KMeans
→
Norm. + RF
Classification
→
Calcul direct
Panier moyen
→
Résultats :
Segment+Churn+Revenu

Règle anti-leakage respectée en production : scaler.transform() UNIQUEMENT (jamais fit), colonnes alignées via reindex(fill_value=0), try/except pour stabilité

Exemples de Prédictions en Production

Freq=1, MT=25£, Tenure=10j
Cluster 0 | Churn: 60% → À RISQUE

Freq=4, MT=190£, Tenure=80j
Cluster 0 | Churn: 49.9% → SURVEILLER

Freq=15, MT=1 200£, Tenure=300j
Cluster 2 | Churn: 18.1% → STABLE

Freq=150, MT=85 000£, Tenure=350j
Cluster 1 | Churn: 7.9% → VIP STABLE

RetailIQ — Customer Analytics

127.0.0.1:5000

Mettre à jour

Conversation

RetailIQ

ML · Random Forest · K=3

Analyse Client en Temps Réel

Segmentation · Prédiction de churn · Estimation du panier moyen

DONNÉES DU CLIENT

FREQUENCY

ex : 8

Nb de commandes

MONETARYTOTAL (€)

ex : 500

Total dépensé

CUSTOMERTENUREDAYS

ex : 365

Ancienneté (jours)

AVGDAYSBETWEENPURCHASES

ex : 20

Délai moyen (jours)

TOTALTRANSACTIONS

ex : 15

Nb total transactions

TOTALQUANTITY

ex : 120

Unités achetées

UNIQUEPRODUCTS

ex : 30

Produits distincts

AGE

ex : 42

Âge du client

SUPPORTTICKETSCOUNT

ex : 2

Aucune analyse lancée

Renseignez les données du client
et cliquez sur Analyser pour obtenir les prédictions

07 Difficultés Rencontrées & Perspectives

Difficultés majeures & Solutions

Data Leakage (v1)

Accuracy=100% grâce à ChurnRiskCategory. Détection lors de l'analyse des features. Suppression obligatoire en étape 1.

Formats de dates hétérogènes

3 formats mélangés (UK/ISO/US). Solution : pd.to_datetime(format='mixed', dayfirst=True) pour prioriser format UK.

Valeurs aberrantes déguisées

SupportTickets (-1, 999) et Satisfaction (-1, 99) : codes erreurs détectés par analyse exploratoire IQR.

Déséquilibre des clusters

Cluster VIP (12 clients) trop petit → K optimal rejeté. Fallback K=3 implémenté automatiquement avec seuil 1%.

Perspectives d'Amélioration

XGBoost / LightGBM

Modèles de boosting généralement plus performants que RandomForest sur données tabulaires → ROC-AUC potentiellement > 0.87.

SHAP — Explicabilité (XAI)

Expliquer individuellement chaque prédiction. Indispensable pour adoption par les équipes métier non-techniques.

KNN / Iterative Imputer

Améliorer l'imputation des 30% NaN sur Age (vs médiane actuelle). KNNImputer utilise les k plus proches voisins.

Déploiement Cloud

Migrer Flask vers Heroku/Azure/AWS. URL publique, monitoring du modèle, retrain automatique sur nouvelles données.

Pipeline sklearn complet

Restructurer preprocessing + modélisation en Pipeline scikit-learn pour reproductibilité et anti-leakage garanti.

Conclusion Générale

La valeur d'un projet Data Science ne réside pas uniquement dans la complexité des algorithmes, mais avant tout dans la rigueur du traitement des données et la pertinence métier des résultats.

4 segments clients identifiés dans la version initiale → 3 segments stables obtenus (fallback K=3) : Ultra-VIP (0.3%, revenu ~110 482£, 53.8% churn) → Alerte stratégique critique

Détection du churn : ROC-AUC = 0.8681, accuracy = 80% — résultats scientifiquement valides après élimination complète du data leakage (ChurnRiskCategory supprimée)

Estimation du revenu : R^2 = 0.8024, MAE = 15.72 £ — précision exploitable en marketing (< 2.5% d'erreur pour les clients standard)

Application Flask déployée : 3 prédictions instantanées (segment + churn + revenu) via interface web accessible sans compétences ML

Rigueur scientifique : mesures anti data leakage appliquées à chaque étape — scaling, imputation, target encoding, suppression des features leakage

Merci pour votre attention